

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ**  
**«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ**  
**СІКОРСЬКОГО»**  
**ФАКУЛЬТЕТ ІНФОРМАТИКИ ТА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ**  
**КАФЕДРА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ**

**ЛАБОРАТОРНА РОБОТА №2 –**  
**ОСНОВНІ ІНСТРУКЦІЇ 32-БІТНОГО ARM ПРОЦЕСОРА**  
**ДЛЯ МІКРОКОНТРОЛЕРІВ**

з дисципліни «Архітектура Комп'ютерів. Частина 3»

Виконав:  
студент групи ІО-31  
Миркін Радислав Ігорович  
номер залікової книжки: 3114

Перевірив:  
Каплунов Артем Володимирович

Київ – 2026

**Мета роботи:** навчитися використовувати асемблерні інструкції ядра ARM Cortex-M4, працювати з процедурами та базово зрозуміти архітектуру ядра. Опанувати синтаксис мови асемблера GAS (GNU Assembly), що є частиною стандартного пакету тулчейну GCC для платформи arm (arm-none-eabi-). Навчитися працювати з відлагоджувачем GDB.

## Теоретична частина

### Основні мікрокоманди асемблера ARM

У ході роботи опрацьовано документацію PM0214 Programming manual та використано наступні групи інструкцій:

- ADD, ADC, SUB, SBC, RSB — додавання та віднімання цілих чисел;
- MUL, MLA, MLS — множення та множення з накопиченням;
- SDIV, UDIV — знакове / беззнакове ділення;
- ASR, LSL, LSR, ROR, RRX — арифметичні та логічні зсуви;
- AND, ORR, EOR — побітові логічні операції;
- PUSH, POP — операції зі стеком (зберігання/відновлення регістрів).

### Директиви асемблера ARM

Програмний код використовує директиви GAS, які керують роботою асемблера, а не транслюються в машинні інструкції:

- .syntax — обирає синтаксис ARM/Thumb (unified — універсальний);
- .cpu — задає цільове ядро (cortex-m4);
- .thumb — активує Thumb-режим (обов'язково для Cortex-M);
- .global — експортує символ для лінкера;
- .type, .size — описують тип і розмір символів;
- .word — виділяє 32-бітне слово даних у пам'яті.

## Типи ARM-інструкцій

У сімействі ARM існує два набори інструкцій: стандартний ARM, що дозволяє швидко виконувати операції (інструкції довжиною від 16 до 64 біт, що ускладнює конвеєр ядра, але дає високу швидкодію), та Thumb — підмножина стандарту, що складається з 16- та 32-бітних інструкцій і дозволяє значно зменшити розмір виконуваної програми. Сімейство Cortex-M4, до якого належить мікроконтролер STM32F407VG, реалізує повний набір Thumb-інструкцій (Thumb-2), але працює виключно у Thumb-режимі.

## Відлагоджувач програм GDB

GDB (GNU Debugger) — відлагоджувач, що дозволяє покроково виконувати програму та слідкувати за станом регістрів і пам'яті. У роботі використовуються такі команди:

- `layout regs` — відображає регістри разом з вихідним кодом;
- `help` — список класів команд відлагоджувача;
- `kill` — примусово зупиняє запущену програму;
- `step` — виконує наступний рядок коду (зайнятий high-level);
- `stepi` — виконує наступну машинну (асемблерну) інструкцію;
- `continue` — продовжує виконання до точки зупину чи кінця;
- `info registers` — виводить значення всіх регістрів процесора;
- `target extended-remote <host>:<port>` — підключення до qemu-сервера;
- `quit` — вихід з GDB.

## Завдання до лабораторної роботи

Написати програму на GAS, яка обчислює функцію за варіантом, та запустити її у відлагоджувачі gdb. Показати регістри з правильним

результатом обчислення. Варіант визначається за номером залікової книжки за формулою  $XXXX \bmod 5$ .

## Визначення варіанту

Номер залікової книжки студента: 3114.

Варіант =  $3114 \bmod 5 = 4$

Функція для варіанту 4 (кускова):

$$f(a,b,c) = \begin{cases} (a \mid b) \cdot c, & \text{якщо } (a \cdot b) \geq 10 \\ (a \cdot b) \cdot c, & \text{якщо } (a \cdot b) < 10 \end{cases}$$

Де " $\mid$ " означає побітове АБО (логічна операція OR над двійковим представленням чисел), " $\cdot$ " — звичайне арифметичне множення. Залежно від того, чи добуток  $a \cdot b$  сягає 10, обирається одна з двох гілок обчислення.

## Обрані тестові значення та очікуваний результат

Для перевірки роботи функції обрано значення:  $a = 4$ ,  $b = 3$ ,  $c = 5$ .

Виконаємо обчислення вручну:

$$\begin{aligned} a \cdot b &= 4 \cdot 3 = 12 & (\geq 10 \rightarrow \text{обираємо першу гілку}) \\ a \mid b &= 4 \mid 3 = 0b100 \mid 0b011 = 0b111 = 7 \\ f &= (a \mid b) \cdot c = 7 \cdot 5 = 35 = 0x23 \end{aligned}$$

Отже, після виконання процедури очікуємо побачити у регістрі  $r0$  значення 35 (шістнадцяткове  $0x23$ ).

## Хід виконання роботи

### 1. Створення файлів проекту

Структура проекту повторює структуру лабораторної роботи №1, але з додатковим файлом lab1.S, що містить обчислювальну процедуру. Робоча директорія — ~/ak3/lab2/.

### 1.1. Файл start.S — таблиця векторів та виклик lab1

Стартовий файл містить таблицю векторів виключень STM32F4 та обробник Reset, який викликає процедуру lab1, а після повернення з неї входить у нескінченний цикл \_loop — це зручно для зупинки програми у відлагоджувачі та перегляду регістрів.

```
.syntax unified
.cpu cortex-m4
//.fpu softvfp
.thumb

.global vtable
.global __hard_reset__

.type vtable, %object
.type __hard_reset__, %function
vtable:
.word __stack_start          /* [0] початкове SP */
.word __hard_reset__ + 1     /* [1] адреса Reset Handler */
.size vtable, .-vtable

__hard_reset__:
    // SP ініціалізовано апаратно з vtable[0]

    bl    lab1                /* виклик процедури обчислення функції */

_loop:
    b     _loop               /* нескінченний цикл для відлагодження */

.size __hard_reset__, .-__hard_reset__
```

Інструкція bl lab1 (Branch with Link) виконує дві дії: зберігає у регістрі lr (link register) адресу повернення (наступну за bl інструкцією) та переходить на мітку lab1. Після виконання процедури lab1 робить bx lr, тобто стрибок за

збереженою адресою — програма повертається до інструкції `_loop` та зациклюється.

## 1.2. Файл `lscript.ld` — скрипт лінкера

Скрипт лінкера ідентичний роботі №1: описує карту пам'яті STM32F407VG — Flash 1 МБ за адресою `0x08000000` та SRAM 128 КБ за адресою `0x20000000`. Символ `__stack_start` обчислюється як кінець RAM.

```
MEMORY
{
    FLASH ( rx ) : ORIGIN = 0x08000000, LENGTH = 1M
    RAM    ( rxw ) : ORIGIN = 0x20000000, LENGTH = 128K
}

__stack_start = ORIGIN(RAM) + LENGTH(RAM);
```

## 1.3. Файл `lab1.S` — обчислення функції за варіантом 4

Це ключовий файл лабораторної роботи. Реалізує процедуру `lab1`, що обчислює кускову функцію  $f(a,b,c)$  для варіанту 4. Використовує інструкції `MUL` (множення), `CMR` (порівняння), `ORR` (побітове АБО), `ITE` (умовне виконання двох інструкцій), а також `PUSH/POP` для роботи зі стеком.

```
.syntax unified
.global lab1

/* --- Тестові вхідні значення для варіанту --- */
#define A    #4      /* перший аргумент функції */
#define B    #3      /* другий аргумент функції */
#define C    #5      /* третій аргумент функції */

.type lab1, %function
lab1:
    push    {lr}      /* зберігаємо адресу повернення у стек */

    /* --- Завантажуємо вхідні значення у регістри --- */
```

```

mov     r0, A                /* r0 = a */
mov     r1, B                /* r1 = b */
mov     r2, C                /* r2 = c */

/* --- Обчислюємо добуток (a · b) для перевірки умови --- */
mul     r3, r0, r1           /* r3 = a · b */

/* --- Перевіряємо умову: (a · b) ≥ 10 ? --- */
cmp     r3, #10              /* порівнюємо r3 з константою 10 */

/* IT-блок: задає умовне виконання двох наступних інструкцій.
 * ITE GE → перша виконується якщо GE, друга – якщо LT. */
ite     ge
orrge   r3, r0, r1           /* (THEN, GE) r3 = a | b */
movlt   r3, r3               /* (ELSE, LT) залишити r3 = a · b */

/* --- Множимо обране проміжне значення на c --- */
mul     r0, r3, r2           /* r0 = r3 · c ← результат функції */

pop     {lr}                 /* відновлюємо lr зі стеку */
bx      lr                   /* повертаємось до викликача */

.size   lab1, .-lab1

```

Опис ключових інструкцій процедури:

- `push {lr}` / `pop {lr}` — стандартний пролог/епілог процедури: оскільки всередині `lab1` ми не викликаємо інших підпрограм, теоретично можна було б не зберігати `lr`, але цей підхід є гарною практикою захисту від змін.
- `mul rd, rn, rm` — множить вміст регістрів `rn` та `rm`, результат поміщає у `rd`; у Cortex-M4 виконується за один такт.
- `cmp r3, #10` — віднімає 10 від `r3` і встановлює прапорці NZCV, але результат не зберігає. Подальші умовні інструкції перевіряють ці прапорці.
- `ite ge` — директива умовного виконання, що описує наступні дві інструкції: якщо умова GE (Greater or Equal) — перша виконується, інакше — друга.



- `orrge r3, r0, r1` — побітове АБО  $r3 = r0 \mid r1$  виконується лише якщо  $a \cdot b \geq 10$ .
- `mul r0, r3, r2` — фінальне множення на `c`, результат у `r0` згідно з ARM Procedure Call Standard є значенням, що повертається з функції.
- `bx lr` — повернення з процедури за збереженою адресою у `lr`.

## 1.4. Файл Makefile

Makefile розширено для збирання двох асемблерних файлів: `start.S` та `lab1.S`. Кожен компілюється окремо в об'єктний файл, після чого обидва лінуються в один `firmware.elf` за скриптом `lscript.ld`.

```
OBJS = start.o lab1.o
```

```
target:
```

```
    $(CC) -x assembler-with-cpp -c -O0 -g3 -mcpu=$(CPU_CC) -mthumb -Wall  
start.S -o start.o
```

```
    $(CC) -x assembler-with-cpp -c -O0 -g3 -mcpu=$(CPU_CC) -mthumb -Wall  
lab1.S -o lab1.o
```

```
    $(CC) $(OBJS) -mcpu=$(CPU_CC) -mthumb -Wall --specs=nosys.specs -  
nostdlib -lgcc -T./lscript.ld -o $(TARGET).elf
```

```
    $(OBJCOPY) -O binary -F elf32-littlearm $(TARGET).elf $(TARGET).bin
```

## 2. Збирання та запуск проекту

### 2.1. Збирання прошивки

Виконано команду `make` у робочій директорії. Кожен `.S`-файл скомпільовано в окремий об'єктний файл, після чого виконано лінування у `firmware.elf` та конвертацію в `firmware.bin`:

```
myrkin-io31@myrkin-io31:~/ak3/lab2$ make  
arm-none-eabi-gcc -x assembler-with-cpp -c -O0 -g3 -mcpu=cortex-m4 -mthumb -  
Wall start.S -o start.o  
arm-none-eabi-gcc -x assembler-with-cpp -c -O0 -g3 -mcpu=cortex-m4 -mthumb -  
Wall lab1.S -o lab1.o
```

```
arm-none-eabi-gcc start.o lab1.o -mcpu=cortex-m4 -mthumb -Wall --
specs=nosys.specs \
    -nostdlib -lgcc -T./lscript.ld -o firmware.elf
arm-none-eabi-objcopy -O binary -F elf32-littlearm firmware.elf firmware.bin
```

Збирання пройшло без помилок та без попереджень. У робочій директорії з'явилися файли start.o, lab1.o, firmware.elf та firmware.bin.

## 2.2. Дизасемблер зібраної процедури lab1

Для перевірки коректності трансляції асемблерного коду в машинні інструкції було виконано команду arm-none-eabi-objdump -d firmware.elf. У виведенні видно, що компілятор згенерував очікувані Thumb-інструкції:

```
08000000 <vtable>:
 8000000:    00 00 02 20 09 00 00 08                ... ....

08000008 <__hard_reset__>:
 8000008:    f000 f801    bl    800000e <lab1>

0800000c <_loop>:
 800000c:    e7fe                b.n    800000c <_loop>

0800000e <lab1>:
 800000e:    b500                push   {lr}
 8000010:    f04f 0004    mov.w r0, #4
 8000014:    f04f 0103    mov.w r1, #3
 8000018:    f04f 0205    mov.w r2, #5
 800001c:    fb00 f301    mul.w r3, r0, r1
 8000020:    2b0a                cmp    r3, #10
 8000022:    bfac                ite    ge
 8000024:    ea40 0301    orrge.w r3, r0, r1
 8000028:    461b                movlt  r3, r3
 800002a:    fb03 f002    mul.w r0, r3, r2
 800002e:    f85d eb04    ldr.w lr, [sp], #4
 8000032:    4770                bx     lr
```

Видно, що таблиця векторів містить адресу 0x20020000 (початкове SP) та 0x08000009 (адреса \_\_hard\_reset\_\_ + 1 для Thumb-режиму). Виклик bl

800000e <lab1> у обробнику Reset переходить на процедуру обчислення функції.

### 2.3. Запуск у qemu

Прошивку завантажено в емулятор qemu-system-gnuarmelipse зі стандартними параметрами для плати STM32F4-Discovery та мікроконтролера STM32F407VG. Прапорці -gdb tcp::1234 -S активують вбудований gdb-сервер та паузять CPU до підключення відлагоджувача:

```
qemu-system-gnuarmelipse --verbose --verbose \  
  --board STM32F4-Discovery --mcu STM32F407VG \  
  -d unimp,guest_errors --image firmware.bin \  
  --semihosting-config enable=on,target=native \  
  -gdb tcp::1234 -S
```

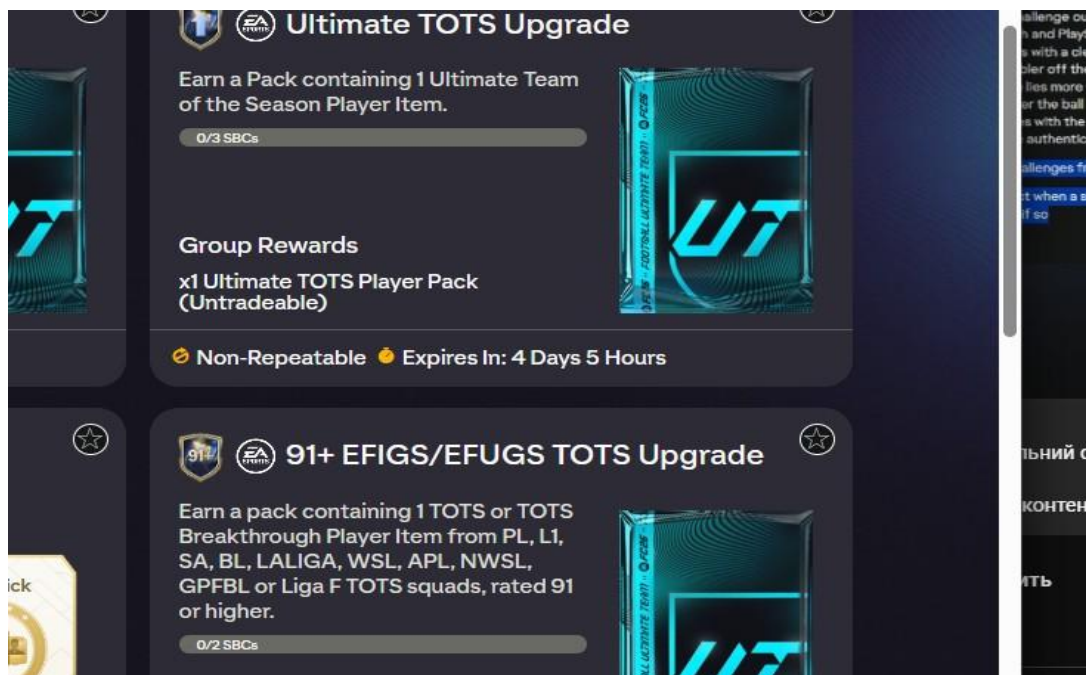


Рис. 1 – Вікно qemu-system-gnuarmelipse з графічним зображенням плати STM32F4-Discovery після завантаження прошивки лабораторної роботи №2

### 2.4. Відлагодження у GDB та перевірка результату

У другому терміналі запущено gdb-multiarch з файлом firmware.elf, виконано підключення до qemu по TCP-порту 1234, встановлено точку зупину на мітці `_loop` та виконано `continue`. Після того, як CPU зупинився у `_loop` (тобто процедура `lab1` вже виконалась і повернула результат), переглянуто стан реєстрів процесора:

```
(gdb) target extended-remote 172.18.224.1:1234
__hard_reset__ () at start.S:43
43      b1      lab1                      /* виклик процедури обчислення функції
*/
(gdb) break _loop
Breakpoint 1 at 0x800000c: file start.S, line 46.
(gdb) continue
Continuing.

Breakpoint 1, __hard_reset__ () at start.S:46
46      b      _loop                      /* нескінченний цикл для відлагодження
*/
(gdb) info registers
r0          0x23                        35
r1          0x3                         3
r2          0x5                         5
r3          0x7                         7
r4          0x0                         0
r5          0x0                         0
r6          0x0                         0
r7          0x0                         0
r8          0x0                         0
r9          0x0                         0
r10         0x0                         0
r11         0x0                         0
r12         0x0                         0
sp          0x20020000                  0x20020000
lr          0x800000d                   134217741
pc          0x800000c                   0x800000c <__hard_reset__+4>
cpsr       0x20000173                   536871283
```

Результат відлагодження повністю співпадає з очікуваним:

- $r0 = 0x23 = 35$  — значення функції, повернуте процедурою `lab1`;
- $r1 = 3$  — друге вхідне значення `b`;
- $r2 = 5$  — третє вхідне значення `c`;

- $r3 = 7$  — проміжний результат  $a \mid b = 4 \mid 3 = 7$  (виконалась THEN-гілка, бо  $a \cdot b = 12 \geq 10$ );
- $sp = 0x20020000$  — вказівник стеку повернувся до початкового стану (push/pop збалансовано);
- $pc = 0x8000000c$  — CPU зупинено в `_loop`, що підтверджує: процедура `lab1` виконалась і повернула керування.

Це доводить, що реалізована процедура коректно обчислює функцію  $f(4, 3, 5) = (4 \mid 3) \cdot 5 = 7 \cdot 5 = 35$  для варіанту 4.

## Висновки

У ході виконання лабораторної роботи №2 було вивчено основні мікрокоманди та директиви асемблера 32-бітного ARM-процесора Cortex-M4, опановано синтаксис мови асемблера GAS (GNU Assembly) та базові команди відлагоджувача GDB. Особлива увага приділена особливостям Thumb-режиму, що використовується сімейством Cortex-M, та інструкціям умовного виконання (IT-блокам).

Створено асемблерну процедуру `lab1`, що реалізує кускову функцію за варіантом 4 ( $3114 \bmod 5 = 4$ ):  $f(a, b, c) = (a \mid b) \cdot c$ , якщо  $a \cdot b \geq 10$ , інакше  $f = (a \cdot b) \cdot c$ . Процедура використовує інструкції `MUL`, `CMR`, `ORR` з умовними варіантами `orrge` / `movlt`, об'єднаними директивою `ITE` — це демонструє ефективний спосіб реалізації умовного виконання без явних інструкцій умовного переходу.

Зібрану прошивку успішно запущено в емуляторі `qemu-system-gnuarmeeclipse` та відлагоджено за допомогою `gdb`. Перевірка регістрів після завершення процедури підтвердила правильність обчислення: для тестових значень  $a = 4$ ,  $b = 3$ ,  $c = 5$  регістр `r0` містить `0x23` (35 десятково), що повністю відповідає теоретично обчисленому результату  $(4 \mid 3) \cdot 5 = 7 \cdot 5 = 35$ .

## **Посилання**

Репозиторій з кодом проекту: <https://github.com/Ratici24/ak3-lab2>

Відеозапис виконання роботи: <https://drive.google.com/file/d/<file-id>/view>